

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Modul Belajar 4

State dan MVVM

Membuat form laporan yang bisa diketik, tetap utuh saat layar diputar, dan logikanya tertata

Muhammad Habib Algifari, S.Kom., M.T.I.

Pendamping Pertemuan 4. Tahun 2026

Komponen	Keterangan
Pertemuan	4
Capaian	CPMK0501, menerapkan konsep pemrograman dalam bentuk aplikasi dengan MVVM yang berfungsi
Waktu belajar	sekitar 5 jam belajar mandiri, ditambah 2,5 jam di kelas

MODUL 4

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: form yang tidak mau diketik
3	Langkah 1: Kode form yang membuat masalah
4	Langkah 2: remember dan mutableStateOf
5	Langkah 3: Tiga cara menulis state, dan recomposition
6	Langkah 4: Masalah kedua, layar diputar
7	Langkah 5: Stateful dan stateless
8	Langkah 6: State turun, event naik
9	Langkah 7: Kenapa butuh ViewModel, dan apa itu MVVM
10	Langkah 8: Satu data class untuk seluruh layar
11	Langkah 9: ViewModel dengan StateFlow
12	Langkah 10: Layar yang membaca ViewModel
13	Latihan mandiri
14	Bawa ke proyekmu
15	Rangkuman
16	Cek pemahaman
17	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini menemani Pertemuan 4. Di Modul 3 kita sudah punya layar daftar laporan. Sekarang kita membuat layar kedua: form laporan baru. Ternyata form jauh lebih menantang daripada daftar, karena isinya berubah setiap kali pengguna mengetik.

Modul ini terdiri dari tiga bagian. Langkah 1 sampai 4 membahas **state**, yaitu data yang berubah dan cara Compose tahu kapan menggambar ulang. Langkah 5 dan 6 membahas cara **mengangkat state**

supaya komponen mudah dipakai ulang. Langkah 7 sampai 10 membahas **ViewModel dan MVVM**, yaitu cara menata state dan logika form di satu tempat yang bertahan saat layar diputar.

Mulai modul ini tidak ada lagi bagian Kotlin seperlunya. Semua bekal bahasa yang kita butuhkan sudah ada di Modul 1 sampai 3.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Mengelola state dengan remember dan mutableStateOf, dan menjelaskan kapan recomposition terjadi	Langkah 1 sampai 4
2	Membuat composable tanpa state yang menerima nilai dan mengirim event	Langkah 5 dan 6
3	Memindahkan state dan logika form ke ViewModel dengan StateFlow	Langkah 7 sampai 10

Capaian ini mendukung **CPMK0501**, dalam bentuk aplikasi dengan MVVM yang berfungsi.

Yang perlu disiapkan

Proyek Laporkampus dengan layar daftar dari Modul 3. Untuk Langkah 7 sampai 10, proyek butuh dependency **lifecycle-viewmodel-compose** dari JetBrains. Proyek hands-on P4 di repo sudah memilikinya, jadi kalian bisa melihat cara mendaftarkannya di `libs.versions.toml` dan `build.gradle.kts` proyek itu.

PENTING Mengingat Modul 2 dan 3

Kita akan memakai data class dengan copy dan Flow dari Modul 2, serta composable, Column, dan lambda `onClick` dari Modul 3. Pola komponen yang hanya melaporkan kejadian di Langkah 8 Modul 3 akan menjadi inti Langkah 5 dan 6 di sini.

PEMBUKA

Masalah pembuka: form yang tidak mau diketik

Minggu lalu kita punya layar daftar. Hari ini kita membuat form laporan baru. Kodenya terlihat benar dan bisa di-build, tetapi pengujian pertama langsung menemukan dua masalah:

Yang dilakukan penguji	Yang terjadi
Mengetik AC di kolom judul	Kolomnya tetap kosong
Mengisi lokasi GKU 2 R305, lalu memutar HP	Isi form hilang semua

TEBAK DULU Coba pikirkan

Kenapa huruf yang diketik tidak muncul? Dan kenapa isi form hilang hanya karena layar diputar?

JAWABAN Arah jawabannya

Masalah pertama soal **state**: Compose tidak tahu ada data yang berubah. Masalah kedua soal **umur state**: tempat penyimpanan datanya ikut hilang saat layar dibuat ulang. Keduanya punya jawaban yang berbeda, dan itulah kerangka modul ini. Jawaban seperti lupa memanggil fungsi atau bug Android Studio memang sering muncul, tetapi bukan penyebabnya.

LANGKAH 1

Kode form yang membuat masalah

Ketik kode berikut di proyek Laporkampus, persis seperti tertulis.

```
commonMain/ui/FormLaporan.kt
```

```

@Composable
fun FormLaporan() {
    var judul = "" // variabel biasa
    Column {
        TextField(
            value = judul,
            onChange = { judul = it },
            label = { Text("Judul") }
        )
        Text("${judul.length} dari 40 karakter")
    }
}

```

Secara Kotlin tidak ada yang salah. TextField menampilkan `judul`, dan setiap kali pengguna mengetik, `onChange` mengubah `judul` menjadi teks baru.

TEBAK DULU

Kita mengetik AC di kolom judul. Apa yang tampil di kolom itu, dan berapa karakter yang tertulis di bawahnya?

JAWABAN

Kolom tetap kosong dan tertulis **0 dari 40 karakter**. Ada dua alasan. Pertama, variabel `judul` memang berubah sesaat di dalam lambda, tetapi Compose tidak diberi tahu, jadi tidak ada yang memicu gambar ulang. Kedua, walaupun `FormLaporan` dipanggil ulang, baris `var judul = ""` mengulang nilainya dari kosong lagi.

PENTING **Buktikan sendiri**

Tambahkan `println(it)` di dalam `onChange`, lalu jalankan dan ketik. Di konsol, huruf yang kalian ketik muncul. Di layar, tidak ada yang berubah. Nilainya berubah, tetapi tampilannya tidak tahu.

ANALOGI **Coretan di kertas bekas**

Variabel biasa di dalam composable seperti mencatat skor pertandingan di kertas bekas, lalu kertasnya langsung dibuang. Skornya memang pernah ditulis, tetapi tidak ada yang melihat, dan saat kalian mengambil kertas baru, skornya kembali nol.

LANGKAH 2

remember dan mutableStateOf

Ubah satu baris saja:

commonMain/ui/FormLaporan.kt

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue

@Composable
fun FormLaporan() {
    var judul by remember { mutableStateOf("") }
    Column {
        TextField(
            value = judul,
            onValueChange = { judul = it },
            label = { Text("Judul") }
        )
        Text("${judul.length} dari 40 karakter")
    }
}
```

Jalankan lagi. Sekarang huruf muncul, dan penghitung karakter ikut berubah.

Bagian	Tugasnya
<code>mutableStateOf("")</code>	Membuat nilai yang diawasi Compose, dengan nilai awal kosong
<code>remember { }</code>	Menyimpan nilai itu supaya tidak kembali kosong saat composable dipanggil ulang
<code>by</code>	Membuat kita bisa membaca dan mengubah nilainya langsung, tanpa menulis <code>.value</code>

Kita butuh keduanya. `mutableStateOf` tanpa `remember` membuat Compose tahu ada perubahan, tetapi nilainya dibuat ulang dari kosong setiap kali dipanggil. `remember` tanpa `mutableStateOf` menyimpan nilainya, tetapi Compose tidak tahu kapan nilainya berubah.

ANALOGI Papan skor

`mutableStateOf` seperti papan skor di lapangan: begitu angkanya diubah, semua penonton langsung melihat. `remember` seperti memastikan papan skor itu tetap terpasang di tempatnya, tidak diganti papan baru yang kosong setiap kali ada gol.

PENTING Import untuk by

Penulisan `by` butuh import `getValue` dan `setValue` dari `androidx.compose.runtime`. Kalau lupa, muncul error yang membingungkan seperti `Property delegate must have a getValue method`.

LANGKAH 3

Tiga cara menulis state, dan recomposition

Tiga cara menulis state yang sama

Saat membaca kode orang lain atau jawaban AI, kalian akan menemukan beberapa gaya menulis state. Kenali saja, tidak perlu dihafal.

Penulisan	Cara mengubah	Catatan
<code>var judul by remember { mutableStateOf("") }</code>	<code>judul = "AC"</code>	Paling ringkas, kita pakai ini
<code>val judul = remember { mutableStateOf("") }</code>	<code>judul.value = "AC"</code>	Sering ada di dokumentasi lama
<code>val (judul, setJudul) = remember { mutableStateOf("") }</code>	<code>setJudul("AC")</code>	Mirip gaya React
<code>val daftar = remember { mutableStateListOf<Laporan>() }</code>	<code>daftar.add(laporan)</code>	Untuk list yang isinya ditambah atau dihapus

Tiga baris pertama sama persis maknanya. Pilih satu gaya dan pakai secara konsisten di proyek kelompok.

Recomposition: hanya yang membaca state

Recomposition adalah Compose memanggil ulang composable yang membaca state yang berubah. Composable yang tidak membaca state itu dilewati. Karena itu, recomposition tetap cepat walaupun terjadi berkali-kali per detik saat pengguna mengetik.

commonMain/ui/FormLaporan.kt

```
@Composable
fun FormLaporan() {
    var judul by remember { mutableStateOf("") }
    Column {
        Text("Laporan baru") // tidak membaca judul
        TextField(value = judul, onValueChange = { judul = it })
        Text("${judul.length} dari 40 karakter") // membaca judul
    }
}
```

Saat judul berubah, TextField dan teks penghitung digambar ulang. Teks Laporan baru tidak, karena tidak membaca judul.

ANALOGI Papan pengumuman

Recomposition seperti petugas yang memperbarui papan pengumuman jurusan. Ia hanya mengganti kertas yang isinya berubah, bukan mencopot dan menempel ulang seluruh papan.

PENTING Akibatnya bagi kita

Badan composable bisa dijalankan berkali-kali. Jangan menaruh pekerjaan berat atau efek samping, seperti memanggil server atau menyimpan ke database, langsung di badan composable. Pekerjaan seperti itu tempatnya di ViewModel, yang kita bahas mulai Langkah 7.

LANGKAH 4

Masalah kedua, layar diputar

Saat layar HP diputar, Android membuat ulang Activity, yaitu wadah tempat seluruh tampilan aplikasi berjalan. Nilai yang hanya disimpan dengan remember ikut hilang bersama wadah lamanya.

Untuk nilai sederhana, ganti remember dengan **rememberSaveable**:

```
commonMain/ui/FormLaporan.kt
var judul by rememberSaveable { mutableStateOf("") }
```

rememberSaveable menyimpan nilainya ke tempat yang selamat dari pembuatan ulang, lalu mengembalikannya ke wadah yang baru.

	Composable dipanggil ulang	Layar diputar
remember	Bertahan	Hilang
rememberSaveable	Bertahan	Bertahan, untuk nilai sederhana
ViewModel	Bertahan	Bertahan, ditambah logika dan data

ANALOGI Pindah kamar kos

Bayangkan pemilik kos memindahkan kalian ke kamar sebelah secara mendadak. Barang yang hanya kalian pegang di tangan (remember) jatuh di jalan. Barang yang dititipkan ke loker resepsionis (rememberSaveable) bisa diambil lagi di kamar baru. Tetapi loker hanya muat barang kecil. Untuk urusan yang lebih besar, kalian butuh pengurus kos yang tetap sama walaupun kamar berganti. Pengurus itulah ViewModel.

PENTING Cara mencoba di emulator

Putar emulator dengan Ctrl dan panah kiri atau kanan, atau tombol putar di panel samping emulator. Target desktop tidak punya rotasi, jadi masalah ini hanya bisa dilihat di Android.

rememberSaveable cukup untuk teks di satu kolom atau posisi tab yang dipilih. Begitu form punya validasi, proses kirim, dan data dari sumber lain, kita butuh tempat yang lebih tepat. Tempat itu adalah ViewModel, di Langkah 7.

LANGKAH 5

Stateful dan stateless

Sebelum ke ViewModel, ada satu kebiasaan penting: memisahkan komponen yang menyimpan state dari komponen yang hanya menampilkan.

Stateful, menyimpan sendiri

```
@Composable
fun KolomJudul() {
    var teks by remember { mutableStateOf("") }
    TextField(
        value = teks,
        onChange = { teks = it }
    )
}
```

KolomJudul menyimpan teksnya sendiri. Nilainya terkunci di dalam. Composable yang memanggilnya tidak bisa membaca, mengisi, atau mengosongkan kolom itu.

Stateless, menerima dan melapor

```

@Composable
fun KolomTeks(
    label: String,
    value: String,
    onValueChange: (String) → Unit
) {
    OutlinedTextField(
        value = value,
        onValueChange = onValueChange,
        label = { Text(label) }
    )
}

```

KolomTeks tidak menyimpan apa pun. Ia menerima nilai yang harus ditampilkan, lalu melaporkan setiap ketikan lewat `onValueChange`. Memindahkan state dari dalam komponen ke pemanggilnya seperti ini disebut **state hoisting**, atau mengangkat state.

ANALOGI Remote TV

Remote TV tidak menyimpan saluran yang sedang tayang. TV yang menyimpannya. Remote hanya mengirim perintah ganti saluran, sehingga satu jenis remote bisa dipakai untuk banyak TV. KolomTeks adalah remote, dan FormLaporan adalah TV.

	Stateful	Stateless
Menyimpan nilai	Di dalam komponen	Di pemanggilnya
Bisa dikosongkan dari luar	Tidak	Bisa
Bisa dipakai ulang	Sulit	Mudah
Mudah diuji	Sulit	Mudah, cukup beri nilai dan cek event

LANGKAH 6

State turun, event naik

Dengan KolomTeks yang stateless, FormLaporan menjadi satu-satunya pemilik state.

```
commonMain/ui/FormLaporan.kt
```

```

@Composable
fun FormLaporan() {
    var judul by remember { mutableStateOf("") }
    var lokasi by remember { mutableStateOf("") }
    Column {
        KolomTeks("Judul", judul) { judul = it }
        KolomTeks("Lokasi", lokasi) { lokasi = it }
        Text("Pratinjau: $judul di $lokasi")
    }
}

```

KolomTeks dipakai untuk judul, lokasi, dan nanti deskripsi. Karena FormLaporan memegang semua state, pratinjau dan tombol kirim membaca sumber yang sama. Lambda terakhir ditulis di luar kurung, seperti yang kita pelajari di Modul 3.

Pola ini disebut **unidirectional data flow**, aliran data satu arah:

Arah	Yang mengalir	Contoh
Turun	State, dari pemilik ke tampilan	FormLaporan mengirim <code>judul</code> ke KolomTeks
Naik	Event, dari tampilan ke pemilik	KolomTeks melaporkan ketikan lewat <code>onValueChange</code>

Pemilik state menerima event, mengubah state, lalu state yang baru turun lagi ke tampilan. Hanya ada satu pemilik dan satu jalan untuk mengubah. Kalau ada bug tampilan, kita cukup memeriksa satu tempat.

CONTOH Contoh sehari-hari

Bayangkan sistem KRS. Mahasiswa tidak mengubah daftar mata kuliahnya sendiri di database. Mahasiswa mengajukan permintaan (event naik), sistem akademik memeriksa dan menyimpan (pemilik mengubah state), lalu halaman KRS menampilkan daftar terbaru (state turun). Kalau ada daftar yang salah, kita tahu harus memeriksa ke mana.

TEBAK DULU

Kalian ingin menambah tombol Kosongkan yang menghapus isi judul dan lokasi. Tombol itu ditaruh di mana, dan apa yang dilakukannya?

JAWABAN

Di FormLaporan, karena di situlah state disimpan. Isinya cukup `judul = ""` dan `lokasi = ""`. KolomTeks tidak perlu diubah sama sekali, karena ia selalu menampilkan nilai yang dikirim kepadanya. Kalau kita memakai KolomJudul yang stateful, tombol ini mustahil dibuat dari luar.

LANGKAH 7

Kenapa butuh ViewModel, dan apa itu MVVM

State di composable sudah bekerja, tetapi punya batas. Bandingkan:

	State di composable	State di ViewModel
Layar diputar	Isi form hilang, kecuali rememberSaveable	Isi tetap, karena ViewModel tidak ikut dibuat ulang
Validasi dan aturan	Bercampur dengan kode tampilan	Di satu kelas, terpisah dari tampilan
Proses kirim	Ikut berhenti saat layar dibuat ulang	Berjalan di viewModelScope sampai selesai
Pengujian	Perlu menjalankan UI	Cukup unit test biasa, kita pakai di Modul 10

ViewModel adalah kelas yang menyimpan state dan logika sebuah layar. Ia hidup lebih lama daripada Activity: Activity bisa dihancurkan dan dibuat ulang saat layar diputar, sedangkan ViewModel tetap sama sampai layarnya benar-benar ditutup.

MVVM di Laporkampus

Dengan ViewModel, kode kita terbagi menjadi tiga lapisan. Pola ini disebut **MVVM**, singkatan dari Model, View, ViewModel.

Lapisan	Di Laporkampus	Tugasnya
View	FormLaporanScreen	Hanya menampilkan uiState dan meneruskan aksi pengguna, tanpa aturan bisnis
ViewModel	FormLaporanViewModel	Menyimpan uiState, memvalidasi, dan memanggil sumber data
Model	Laporan dan sumber data	Data class dan sumber data dari Modul 2, nanti diganti server dan database

Alirannya sama dengan Langkah 6. Event naik dari View ke ViewModel, ViewModel meminta data ke Model, lalu state turun dari ViewModel ke View. Bedanya hanya pemilik state yang pindah, dari composable ke ViewModel.

ANALOGI Rumah makan

View adalah pelayan yang mencatat pesanan dan mengantar makanan, tanpa memasak. ViewModel adalah kepala dapur yang memutuskan pesanan mana yang bisa dibuat dan mengatur urutannya. Model adalah gudang bahan. Pelayan boleh berganti sif, tetapi kepala dapur tetap ingat pesanan yang sedang dikerjakan.

TEBAK DULU

Aturan bahwa judul laporan minimal tiga huruf ditaruh di lapisan mana?

JAWABAN

Di **ViewModel**. Itu aturan bisnis, bukan urusan tampilan. View hanya menampilkan hasil keputusannya, misalnya tombol Kirim yang aktif atau tidak.

LANGKAH 8

Satu data class untuk seluruh layar

Daripada banyak variabel state yang terpisah, kita kumpulkan semua yang dibutuhkan layar form ke dalam satu data class.

commonMain/ui/FormLaporanUiState.kt

```
data class FormLaporanUiState(
    val judul: String = "",
    val lokasi: String = "",
    val prioritas: Int = 3,
    val sedangMengirim: Boolean = false,
    val pesan: String? = null
) {
    val bisaKirim: Boolean
        get() = judul.isNotBlank() &&
            lokasi.isNotBlank() && !sedangMengirim
}
```

Semua yang dibutuhkan layar ada di satu tempat, dan setiap nilai punya bawaan. `pesan` bertipe nullable karena pesan hanya ada setelah laporan terkirim.

`bisaKirim` adalah properti dengan `get()`. Nilainya dihitung dari nilai lain setiap kali dibaca, tidak disimpan sendiri. Jadi mustahil tombol Kirim aktif padahal judul masih kosong.

ANALOGI Dasbor motor

UiState seperti dasbor motor. Semua yang perlu dilihat pengendara ada di satu panel: kecepatan, bensin, lampu sein. Lampu peringatan bensin tidak dinyalakan manual. Ia menyala sendiri berdasarkan isi tangki. Begitu pula bisaKirim, ia dihitung dari isi judul dan lokasi.

PENTING Prinsip state seminimal mungkin

Simpan hanya yang tidak bisa dihitung, lalu hitung sisanya. Setiap nilai yang disimpan doblel adalah peluang dua nilai itu tidak sinkron.

LANGKAH 9

ViewModel dengan StateFlow

```
commonMain/ui/FormLaporanViewModel.kt
```

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

class FormLaporanViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(FormLaporanUiState())
    val uiState = _uiState.asStateFlow()

    fun ubahJudul(teks: String) =
        _uiState.update { it.copy(judul = teks.take(40)) }

    fun ubahLokasi(teks: String) =
        _uiState.update { it.copy(lokasi = teks) }

    fun kirim() {
        if (!_uiState.value.bisaKirim) return
        viewModelScope.launch {
            _uiState.update { it.copy(sedangMengirim = true) }
            delay(800) // Modul 6: diganti panggilan server
            _uiState.value = FormLaporanUiState(pesan = "Laporan terkirim")
        }
    }
}
```

Bagian	Kenapa
<code>_uiState</code>	Private, hanya ViewModel yang boleh mengubah
<code>uiState</code>	Publik dan hanya bisa dibaca oleh layar
<code>update { it.copy() }</code>	Mengganti objek lama dengan salinan baru, tidak mengubah objek lama
<code>teks.take(40)</code>	Aturan maksimal 40 karakter dijaga di ViewModel
<code>viewModelScope</code>	Coroutine otomatis berhenti saat ViewModel dibersihkan

StateFlow adalah Flow dari Modul 2 yang selalu punya nilai terakhir. Siapa pun yang mulai mengamati langsung menerima nilai terkini, lalu setiap perubahan berikutnya.

ANALOGI Rekening bank

MutableStateFlow yang private seperti saldo di sistem bank: hanya bank yang boleh mengubahnya. StateFlow yang publik seperti aplikasi mobile banking: kalian bisa melihat saldo terbaru kapan saja, tetapi tidak bisa mengetik angka saldo sendiri. Untuk mengubah saldo, kalian mengirim permintaan transfer, sama seperti layar memanggil `vm.kirim()`.

Sifat penting StateFlow

Jalankan kode ini di Kotlin Playground untuk melihat cara StateFlow bekerja tanpa Android Studio.

Main.kt

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.MutableStateFlow

fun main() = runBlocking {
    val state = MutableStateFlow("")
    val ui = launch { state.collect { println("UI menggambar: '$it'") } }
    delay(100); state.value = "AC"
    delay(100); state.value = "AC"           // nilainya sama
    delay(100); state.value = "AC mati"
    delay(100); ui.cancel()
}
```

TEBAK DULU

Berapa kali baris UI menggambar tercetak?

JAWABAN

Tiga kali, bukan empat.

Keluaran

```
UI menggambar: ''
UI menggambar: 'AC'
UI menggambar: 'AC mati'
```

StateFlow tidak memancarkan ulang nilai yang sama dengan nilai sebelumnya. Inilah juga alasan tampilan tidak digambar ulang kalau state tidak benar-benar berubah.

LANGKAH 10

Layar yang membaca ViewModel

commonMain/ui/FormLaporanScreen.kt

```
import androidx.compose.runtime.collectAsState
import androidx.lifecycle.viewmodel.compose.viewModel

@Composable
fun FormLaporanScreen(
    vm: FormLaporanViewModel = viewModel { FormLaporanViewModel() }
) {
    val state by vm.uiState.collectAsState()
    Column(modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(12.dp))
    {
        KolomTeks("Judul", state.judul, vm::ubahJudul)
        KolomTeks("Lokasi", state.lokasi, vm::ubahLokasi)
        Button(onClick = vm:: kirim, enabled = state.bisaKirim) {
            Text(if (state.sedangMengirim) "Mengirim" else "Kirim")
        }
        state.pesan?.let { Text(it) }
    }
}
```

Bagian	Artinya
<code>viewModel { FormLaporanViewModel() }</code>	Mengambil ViewModel yang sudah ada, atau membuatnya kalau belum ada. Di commonMain, cara membuatnya wajib ditulis
<code>collectAsState()</code>	Mengubah StateFlow menjadi state Compose, sehingga layar ikut berubah
<code>vm:: ubahJudul</code>	Referensi fungsi, sama artinya dengan <code>{ vm.ubahJudul(it) }</code>
<code>state.pesan?.let { Text(it) }</code>	Menampilkan pesan hanya kalau tidak null

Perhatikan bahwa KolomTeks dari Langkah 5 dipakai tanpa diubah sedikit pun. Inilah hasil dari membuatnya stateless.

TEBAK DULU

Setelah mengetik AC mati, layar diputar. Apa isi kolom judul sekarang? Lalu, apa yang terjadi kalau tombol Kirim ditekan dua kali dengan cepat?

JAWABAN

Kolom judul tetap berisi **AC mati**, karena state ada di ViewModel yang tidak ikut dibuat ulang. Tekanan kedua tidak berbuat apa-apa. Saat tekanan pertama, sedangMengirim menjadi true, sehingga bisaKirim bernilai false dan tombol nonaktif. Pemeriksaan `if (!bisaKirim)` return di awal fungsi kirim menjadi pengaman kedua.

Menyambungkan form dengan layar daftar

Supaya laporan baru muncul di layar daftar Modul 3, kita butuh tempat penyimpanan yang dibaca oleh keduanya. Untuk sementara kita pakai objek sederhana berisi MutableStateFlow.

commonMain/data/PenyimpananLaporan.kt

```
object PenyimpananLaporan {
    private val _daftar = MutableStateFlow(contoh) // tiga laporan dari Modul 3
    val daftar: StateFlow<List<Laporan>> = _daftar.asStateFlow()

    fun tambah(judul: String, lokasi: String) = _daftar.update { lama →
        lama + Laporan(lama.size + 1, judul, lokasi, null)
    }
}
```

Lalu di ViewModel, ganti baris `delay(800)` dengan:

commonMain/ui/FormLaporanViewModel.kt

```
delay(800)
PenyimpananLaporan.tambah(_uiState.value.judul, _uiState.value.lokasi)
```

Dan di layar daftar, baca daftarnya dengan `collectAsState`:

commonMain/App.kt

```
@Composable
fun App() = MaterialTheme {
    var diForm by remember { mutableStateOf(false) }
    val daftar by PenyimpananLaporan.daftar.collectAsState()
    Column {
        Button(onClick = { diForm = !diForm }) {
            Text(if (diForm) "Lihat daftar" else "Lapor")
        }
        if (diForm) FormLaporanScreen() else DaftarLaporan(daftar, onPilih = { })
    }
}
```

Perpindahan layar dengan `if` ini hanya sementara. Kelemahannya akan terasa saat tombol kembali ditekan, dan kita menggantinya dengan navigasi sungguhan di Modul 5.

Lima kebiasaan yang kita pegang

Kelima kebiasaan berikut yang membedakan level Sangat baik dan Baik di observasi praktik minggu ini.

Kebiasaan	Di kode kita
State seminimal mungkin	bisaKirim dihitung, tidak disimpan
Satu sumber kebenaran	Isi form hanya ada di FormLaporanViewModel
Hanya pemilik yang mengubah	MutableStateFlow private, StateFlow publik
Ubah dengan salinan	<code>update { it.copy(...) }</code> , tidak mengubah objek lama
Tanpa efek samping di composable	Mengirim data lewat ViewModel, tidak di badan composable

PRAKTIK

Latihan mandiri

Ada tiga latihan di folder P4 State Management MVVM Hands-on. Menu di App.kt berpindah antara latihan dan solusi. Latihan 3 yang diamati.

Latihan	Berkas	Yang dilatih
1	Handson1Counter.kt	Counter dengan tombol tambah, kurang yang nonaktif di nol, dan reset
2	Handson2RegistrationForm.kt	LabeledTextField tanpa state untuk nama dan email, lalu pratinjau
3	Handson3TodoApp.kt	TodoViewModel dengan StateFlow untuk menambah dan mencentang todo

CONTOH Petunjuk latihan 1

Satu state Int dengan remember dan mutableStateOf. Tombol kurang memakai `enabled = angka > 0`, persis seperti tombol Kirim yang memakai `bisaKirim`. Jangan menyimpan status aktif tombol sebagai state terpisah, hitung saja.

CONTOH Petunjuk latihan 2

LabeledTextField adalah KolomTeks di Langkah 5 dengan nama lain. Ia tidak boleh punya remember di dalamnya. State nama dan email disimpan di composable form, lalu pratinjau membaca dari sana.

CONTOH Petunjuk latihan 3

Ikuti pola Langkah 8 dan 9. Buat dulu data class untuk satu todo dan untuk UiState layar. Di TodoViewModel, `_uiState` private dan `uiState` publik. Mencentang todo berarti mengganti satu item dengan salinannya: `daftar.map { if (it.id == id) it.copy(selesai = !it.selesai) else it }`. Layar hanya membaca state dan memanggil fungsi ViewModel.

Yang diamati di latihan 3

Level	Skor	Yang terlihat
Sangat baik	85 sampai 100	TodoViewModel dengan StateFlow, UI hanya membaca state, tambah dan centang berjalan

Level	Skor	Yang terlihat
Baik	70 sampai 84	MVVM berjalan, sebagian logika masih di composable
Cukup	55 sampai 69	Hanya memakai remember, belum ada ViewModel
Kurang	di bawah 55	Aplikasi belum bisa dipakai

Observasi dikerjakan individu dengan bobot 4% dari nilai akhir, untuk CPMK0501. Buktinya: push latihan ke repository latihan pribadi, lalu kirim tautan commit di LMS sebelum kelas selesai.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
Tampilan tidak berubah saat diketik	Variabel biasa, bukan state	<code>remember { mutableStateOf() }</code>
Nilai kembali kosong	<code>mutableStateOf</code> tanpa <code>remember</code>	Bungkus dengan <code>remember</code>
Unresolved reference atau delegate error pada by	Import belum ada	Import <code>getValue</code> dan <code>setValue</code> dari <code>androidx.compose.runtime</code>
<code>collectAsState</code> not found	Import belum ada	<code>import androidx.compose.runtime.collectAsState</code>
ViewModel atau <code>viewModel</code> not found	Dependency belum ada	Tambahkan <code>lifecycle-viewmodel-compose</code> dari JetBrains
Tombol tidak pernah aktif	<code>bisaKirim</code> tidak pernah <code>true</code>	Cetak <code>state</code> dengan <code>println</code> , periksa nilai yang dipakai <code>bisaKirim</code>

PRAKTIK

Bawa ke proyekmu

Ini layar kedua dari lima layar wajib Bank Proyek. Bagi kerja di kelompok: satu anggota mengerjakan `UiState` dan `ViewModel`, satu anggota layar form, satu anggota tempat penyimpanan data.

- UiState form.** Data class berisi kolom entitas utama tema kalian dan properti `bisaSimpan` yang dihitung.
- ViewModel.** `MutableStateFlow` private, `StateFlow` publik, dan validasi sebelum simpan.
- KolomTeks dipakai ulang.** Satu composable tanpa state untuk semua kolom isian.
- Daftar ikut bertambah.** Tempat penyimpanan menyimpan daftar dalam `MutableStateFlow`, dan layar daftar membacanya.

CONTOH Contoh dari LaporanKampus

FormLaporanUiState, FormLaporanViewModel, KolomTeks, dan PenyimpananLaporan.
Laporan baru langsung muncul di layar daftar Modul 3.

Checkpoint minggu 4

No	Yang diselesaikan
1	Layar form entitas utama tema, memakai KolomTeks tanpa state
2	UiState dan ViewModel dengan StateFlow, simpan hanya bila valid
3	Data baru masuk tempat penyimpanan dan tampil di layar daftar

Push ke repository kelompok. README memuat tangkapan layar form dan daftar setelah data ditambah. Checkpoint tidak masuk nilai, tetapi menjadi bekal tes lisan di pertemuan 12 sampai 16. Form ini dipakai lagi minggu depan sebagai tujuan navigasi dari layar daftar.

PENUTUP

Rangkuman

State adalah data yang berubah dan diawasi Compose. `mutableStateOf` membuat Compose tahu ada perubahan, dan `remember` menjaga nilainya saat composable dipanggil ulang. Kita butuh keduanya.

Recomposition memanggil ulang hanya composable yang membaca state yang berubah. Karena bisa terjadi berkali-kali, badan composable tidak boleh berisi efek samping.

Saat layar diputar, nilai di remember hilang. `rememberSaveable` menyelamatkan nilai sederhana, sedangkan ViewModel menyelamatkan state dan logika sekaligus.

State hoisting memindahkan state ke pemanggil, sehingga komponen menjadi stateless: menerima nilai dan melaporkan event. Pola **state turun, event naik** membuat hanya ada satu pemilik state.

MVVM membagi kode menjadi View yang menampilkan, ViewModel yang memutuskan, dan Model yang menyimpan data. ViewModel menyimpan satu data class `UiState` dalam `MutableStateFlow` yang private, membukanya sebagai `StateFlow`, dan mengubahnya dengan `update { it.copy() }`. Layar membacanya dengan `collectAsState()`.

PENUTUP

Cek pemahaman

Jawab tanpa melihat modul dulu, lalu cocokkan dengan kunci di bawahnya.

1. Kenapa `var judul = ""` di dalam composable membuat kolom tidak bisa diketik?
2. Apa yang terjadi kalau kita menulis `var judul by mutableStateOf("")` tanpa `remember`?
3. Isi kolom hilang saat layar diputar padahal sudah memakai `remember`. Apa dua cara memperbaikinya?
4. Apa yang dikirim parent ke `KolomTeks`, dan apa yang dikirim `KolomTeks` ke parent?
5. Kenapa `bisaKirim` ditulis dengan `get()` dan tidak disimpan sebagai properti biasa?
6. Kenapa `uiState` yang publik bertipe `StateFlow`, bukan `MutableStateFlow`?
7. Sebuah `MutableStateFlow` dibuat dengan nilai awal kosong, lalu diberi nilai "A", "A" lagi, dan "B", dengan jeda di antaranya. Selain nilai awal, berapa kali collector menerima nilai?

JAWABAN Kunci jawaban

1. Tidak ada state yang diawasi `Compose`, jadi tidak ada recomposition. Walaupun composable dipanggil ulang, nilainya diulang dari kosong.
2. `Compose` tahu ada perubahan, tetapi setiap kali composable dipanggil ulang, state baru dibuat dengan nilai kosong. Hasilnya kolom tetap kosong.
3. Memakai `rememberSaveable` untuk nilai sederhana, atau memindahkan state ke `ViewModel`.
4. Parent mengirim nilai (value) yang harus ditampilkan. `KolomTeks` mengirim event `onValueChange` berisi teks baru.
5. Supaya selalu dihitung dari judul, lokasi, dan `sedangMengirim`. Dengan begitu tombol tidak mungkin aktif saat isian belum lengkap.
6. Supaya hanya `ViewModel` yang bisa mengubah state. Layar hanya bisa membaca dan meminta perubahan lewat fungsi `ViewModel`.
7. Dua kali, yaitu "A" dan "B". Nilai "A" yang kedua sama dengan sebelumnya, jadi tidak dipancarkan ulang.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

Sumber	Untuk apa
developer.android.com, State and Jetpack Compose	Cara Compose mengawasi state
Codelab State in Jetpack Compose	Latihan state dan state hoisting langkah demi langkah
developer.android.com, Guide to app architecture	Gambaran besar lapisan View, ViewModel, dan Model
kotlinlang.org, Multiplatform ViewModel	ViewModel di commonMain untuk Android, iOS, dan desktop
kotlinlang.org, StateFlow and SharedFlow	Sifat StateFlow

CONTOH Masalah Pertemuan 5

Daftar dan form sudah ada. Bagaimana pindah dari daftar ke detail satu laporan sambil membawa id-nya, lalu kembali tanpa kehilangan posisi gulir? Coba tekan tombol kembali di Android saat form terbuka dengan `if` di App. Apa yang terjadi? Pastikan TodoApp latihan 3 sudah di-push ke repository latihan pribadi.